

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning (BL 5)

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks: 20

Annexure A

Comparative Analysis

Code of Conduct:

I V.Madhulika (192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student

Q1. Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.

Answer:

In a medical diagnosis system, KNN stores every past patient record as a purely numeric/categorical feature vector and classifies a new patient by majority vote among the K closest records using a fixed distance metric such as Euclidean distance — it has no concept of 'reasoning' beyond geometric closeness. Case-Based Learning also stores past patient encounters, but each case is richer: it includes the problem description, the reasoning/solution path a clinician followed, and the outcome, and retrieval can use structured, domain-informed similarity measures (e.g. weighting symptom overlap or matching diagnostic categories more heavily than minor demographic differences) rather than a single flat distance function. In terms of memory usage, both are memory-intensive since they retain the full dataset, but CBL typically stores richer per-case metadata (justifications, outcomes) so its memory footprint per record is larger. For prediction, KNN purely aggregates neighbour labels by vote or average, whereas CBL explicitly reuses and adapts the retrieved case's solution to the new situation and can revise/store the outcome back into the case base, giving it a built-in learning loop that plain KNN lacks. CBL is therefore better suited when explainability and reuse of expert reasoning matter, while KNN is simpler and faster to implement when only classification accuracy is needed.

Q2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

Answer:

For approximating a noisy, non-linear function such as sensor-based machine output over time, Locally Weighted Regression solves an entirely new weighted regression at every query point using nearby training data, giving it maximal locality — its behaviour can differ sharply between neighbouring queries if the local data supports it — but this also makes it computationally expensive at prediction time, since each query requires solving a (weighted) least-squares problem from scratch over the training set. RBF networks instead fix a smaller number of Gaussian centres once during training and combine their activations with learned weights; prediction is then just a fast forward pass through these fixed basis functions, making RBF far cheaper at inference time and better suited to real-time or high-throughput applications. In terms of smoothness, RBF networks tend to produce smoother, more continuous predictions across the whole input space because the fixed Gaussian basis functions overlap and blend gradually, whereas LWR can produce predictions that are locally very accurate but exhibit more visible transitions between regions with different amounts of local training support. Overall, LWR trades computational cost for maximal local accuracy, while RBF networks trade a small amount of local precision for speed and smoothness.

Q3. Analyze the differences between K-Nearest Neighbour and Radial Basis Function models in classification problems. Provide an example and compare decision boundaries, training time, and generalization ability.

Answer:

For a classification example such as detecting network intrusions from traffic features, KNN has effectively zero training time — it simply stores the labelled data — but its decision boundary is a jagged, piecewise pattern implicitly defined by the Voronoi regions of the stored points, and every prediction requires scanning the full training set, making it slow at inference for large datasets. RBF networks require an explicit training phase (selecting/clustering centres and solving for the output-layer

weights), so training time is higher, but once trained the decision boundary is a smooth combination of overlapping Gaussian regions, which is often more robust to noisy individual data points than KNN's boundary that can be distorted by a single mislabelled neighbour. In terms of generalisation, KNN's performance depends heavily on the chosen K and can overfit at $K=1$ or underfit at very large K , while RBF's generalisation depends on the number of centres and their width — too many narrow centres overfit similarly to a small K in KNN, while too few wide centres underfit. Overall, RBF tends to generalise more smoothly and predict faster once trained, while KNN is simpler to set up and update but scales poorly with data size at prediction time.

Q4. Compare Case-Based Learning and Locally Weighted Regression using a practical example such as recommendation systems. Discuss how each approach handles new data, adaptability, and computational efficiency.

Answer:

In a product recommendation setting, Case-Based Learning retrieves the most similar past purchasing 'case' (a customer profile and the products/strategy that worked for them) and reuses that solution, explicitly storing the outcome of each new recommendation back into the case base — this makes CBL naturally adaptive, since the case base grows and improves with every new confirmed outcome, and it can also incorporate qualitative, non-numeric context (e.g. customer feedback text) more naturally than a purely numeric regression. Locally Weighted Regression, by contrast, would typically be used to predict a numeric quantity such as expected spend or click-through probability, fitting a fresh weighted regression from nearby historical data points for every new prediction; it adapts to new data simply by including it in the pool of points available for future local fits, without any explicit 'reasoning/reuse/revise' cycle. Computationally, CBL's cost depends on how similarity search and case retrieval are indexed, but the retrieval step alone is comparable in cost to KNN, whereas LWR must additionally solve a regression at every query on top of the neighbour search, making LWR more computationally expensive per prediction. In practice, CBL suits problems needing explainable, reusable expert-like decisions, while LWR suits smooth numeric prediction tasks where local curvature matters more than symbolic reasoning.

Q5. Compare Decision Tree Learning and K-Nearest Neighbour (KNN) using a classification problem such as customer churn prediction. Analyze differences in model interpretability, training time, handling of noisy data, and decision-making process.

Answer:

For customer churn prediction, a Decision Tree learns an explicit set of if-then splits (e.g. "if tenure < 6 months and monthly charges > \$70, then likely to churn"), which is highly interpretable and easy to present to business stakeholders, and once trained, prediction is a fast traversal down the tree. KNN, by contrast, has no explicit trained model — it stores all customer records and predicts by majority vote among the nearest historical customers — making it far less interpretable, since there is no simple rule that explains why a prediction was made beyond 'these are the most similar past customers'. In terms of training time, Decision Trees require an upfront training pass to compute the best splits (using Gini index or information gain), while KNN has no real training cost but shifts that cost to every prediction, which becomes slow as the customer base grows. Decision Trees can be sensitive to noisy data and prone to overfitting a single tree to spurious splits (though this is mitigated by pruning or ensembles like Random Forest), whereas KNN's predictions are naturally smoothed by averaging over K neighbours, making it somewhat more robust to individual noisy records, though still sensitive to irrelevant or unscaled features distorting distance calculations. On large, high-dimensional customer datasets, Decision Trees generally scale and generalise better, while KNN becomes computationally expensive and can suffer from the curse of dimensionality.

Q6. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data.

Answer:

For spam detection using word-frequency features, Naïve Bayes assumes that each word's presence is conditionally independent of every other word given the class (spam or not spam) — an assumption that is technically false (words co-occur meaningfully) but works well in practice because the model only needs the relative ranking of class probabilities to be correct, not their exact values. Logistic Regression makes no independence assumption; it directly learns a set of weighted coefficients for each word feature that best separate the two classes, which can capture some interactions between correlated words implicitly through the combined weighting, at the cost of requiring an iterative optimisation (e.g. gradient descent) rather than Naïve Bayes' simple closed-form probability counting. Computationally, Naïve Bayes is extremely efficient to train — a single pass over the data to count word/class frequencies — while Logistic Regression requires multiple iterations to converge, making Naïve Bayes attractive for very large or frequently retrained email corpora. On high-dimensional, sparse text data (large vocabularies), Naïve Bayes tends to be surprisingly robust and fast despite its unrealistic independence assumption, while Logistic Regression, especially with regularisation (L1/L2), can achieve higher accuracy by learning more nuanced feature weights but requires careful tuning to avoid overfitting on the large feature space. In practice, Naïve Bayes is often used as a fast baseline, with Logistic Regression preferred when maximising accuracy is worth the extra training cost.

Q7. Analyze the differences between Support Vector Machines (SVM) and KNN in classification tasks. Use an example dataset to compare decision boundaries, scalability, sensitivity to noise, and computational complexity. Discuss how kernel functions in SVM improve performance for non-linear problems.

Answer:

For a classification example such as separating genuine and fraudulent transactions, SVM explicitly solves an optimisation problem to find the maximum-margin hyperplane separating the classes, producing a decision boundary defined only by the support vectors closest to the margin, which tends to generalise well and is relatively robust to noise since it focuses on the boundary rather than every training point. KNN's decision boundary, by contrast, is an implicit, jagged partition determined by the local density of every stored point, meaning individual noisy or mislabelled points near the boundary can visibly distort predictions in their vicinity. In terms of scalability, SVM training is more computationally expensive upfront (particularly with non-linear kernels on large datasets) but prediction is fast once the support vectors are found, whereas KNN needs no training but every prediction requires a distance computation against the full (or a large portion of the) training set, so KNN scales worse as the dataset grows for repeated predictions. Kernel functions (such as the RBF/Gaussian kernel) allow SVM to implicitly map data into a higher-dimensional space where a linear boundary can separate classes that are not linearly separable in the original feature space, giving SVM strong performance on non-linear problems without the explicit computational cost of the transformation (the 'kernel trick'), a capability KNN does not need since its neighbour-based approach is naturally non-linear but at the cost of scalability rather than optimisation complexity.

Q8. Compare K-Means Clustering and Hierarchical Clustering using a dataset with unknown structure. Analyze differences in cluster formation, computational cost, scalability, and interpretability. Discuss how each method handles varying cluster shapes and the impact of choosing the number of clusters.

Answer:

On a dataset with unknown structure, K-Means requires the number of clusters k to be specified in advance and partitions data by iteratively assigning points to the nearest centroid and updating centroids, which works well for roughly spherical, similarly-sized clusters but performs poorly on elongated or irregularly-shaped clusters since it relies on Euclidean distance to a single centroid. Hierarchical Clustering does not require k upfront — it builds a full dendrogram of nested clusters by successively merging (or splitting) groups, and the analyst can choose the number of clusters afterward by cutting the dendrogram at an appropriate height, which is especially useful for exploratory analysis of unknown structure. Computationally, K-Means is much cheaper — roughly linear in the number of points per iteration — making it scale well to large datasets, whereas Hierarchical Clustering (agglomerative) typically has quadratic or worse time and memory complexity, limiting its practicality on very large datasets. In terms of interpretability, the dendrogram produced by Hierarchical Clustering gives rich insight into the nested structure and relative distances between clusters at every level, while K-Means only outputs a flat final partition. Hierarchical Clustering with single or average linkage can also capture more varied cluster shapes than K-Means' inherently spherical bias, though it is more sensitive to noise and outliers depending on the linkage method chosen. Choosing k poorly in K-Means directly distorts the resulting clusters, whereas Hierarchical Clustering defers that choice, offering more flexibility when the right number of clusters is not known in advance.

Q9. Compare Genetic Algorithms and Gradient Descent Optimization for solving optimization problems. Analyze differences in convergence behavior, ability to escape local minima, computational cost, and suitability for different types of objective functions.

Answer:

Gradient Descent optimises a function by iteratively moving in the direction of steepest descent using the gradient, which converges quickly and efficiently for smooth, differentiable, ideally convex objective functions, but it can get trapped in local minima (or saddle points) for non-convex functions since it only uses local gradient information and has no mechanism to explore distant regions of the search space. Genetic Algorithms optimise using a population-based search — selection, crossover, and mutation across many candidate solutions simultaneously — which gives them a much stronger ability to escape local minima and explore a wider region of the search space, especially for objective functions that are non-differentiable, discontinuous, or highly multi-modal, where gradient information is unavailable or misleading. This robustness comes at a computational cost: GAs generally require many more function evaluations across a population and many generations to converge, making them considerably slower and more resource-intensive than gradient descent when a well-behaved gradient is actually available. Gradient descent is therefore preferred for training most machine learning models (where the loss function is differentiable and gradients can be computed efficiently via backpropagation), while genetic algorithms are more suitable for combinatorial optimisation, hyperparameter search, or problems where the objective function is a black box with no usable gradient.

Q10. Compare Principal Component Analysis (PCA) and Feature Selection techniques for dimensionality reduction. Analyze differences in information preservation, computational complexity, interpretability, and impact on model performance.

Answer:

PCA reduces dimensionality by constructing new features (principal components) as linear combinations of all original features, chosen to capture the maximum variance in the data; this generally preserves more overall information from the dataset for a given number of output dimensions, but the resulting components are abstract linear blends that are hard to interpret in terms of the original, domain-

meaningful variables. Feature Selection techniques (such as filter methods based on correlation/mutual information, or wrapper methods like recursive feature elimination) instead choose a subset of the original features to keep, discarding the rest entirely; this preserves full interpretability, since the retained features are still the original, meaningful variables, but it can lose information contained in the discarded features that PCA would have partially retained by blending it into components. Computationally, simple filter-based feature selection is typically cheaper than PCA (which requires an eigen-decomposition or SVD of the covariance matrix), while wrapper-based feature selection that repeatedly retrains a model to evaluate subsets can be considerably more expensive than PCA. In terms of downstream model performance, PCA often yields comparable or better accuracy when many original features are correlated and redundant (since it optimally compresses that redundancy), whereas feature selection can outperform PCA when only a handful of features are genuinely predictive and the rest are noise, since PCA components would otherwise mix that noise into the retained dimensions.

Q11. Compare Random Forest and Decision Tree algorithms using a classification problem such as credit risk assessment. Analyze differences in overfitting control, accuracy, interpretability, and robustness to noisy data.

Answer:

For credit risk assessment, a single Decision Tree is highly interpretable — the exact sequence of financial-feature splits leading to a risk decision can be read directly off the tree, which is valuable for regulatory explainability — but a single tree grown to full depth easily overfits the training data, memorising noise and specific applicant quirks rather than learning generalisable risk patterns. Random Forest addresses this by training many decision trees on bootstrapped samples of the data with randomly restricted feature subsets at each split, then averaging (or majority-voting) their predictions; this ensemble approach substantially reduces overfitting and variance compared to a single tree, typically yielding higher and more stable accuracy on unseen applicants. The trade-off is interpretability: a Random Forest's prediction is the aggregate of potentially hundreds of trees, so there is no single readable rule path, and explaining an individual decision requires post-hoc tools such as feature-importance scores or SHAP values rather than direct tree traversal. Random Forest is also considerably more robust to noisy or mislabelled records in the training data, since the errors introduced by any one noisy tree tend to be averaged out across the ensemble, whereas a single Decision Tree can have its structure significantly altered by a small number of noisy training examples near a split point.

Q12. Compare Linear Regression and Polynomial Regression for predicting housing prices. Analyze differences in model complexity, ability to capture non-linear relationships, risk of overfitting, and interpretability.

Answer:

For predicting housing prices from features such as square footage, Linear Regression fits a single straight-line relationship between each feature and price, which is simple, fast to train, and highly interpretable (each coefficient directly states how much price changes per unit increase in a feature), but it cannot capture non-linear effects such as diminishing returns on very large houses or accelerating price increases in premium size brackets. Polynomial Regression extends the model by adding polynomial terms (e.g. square footage squared, cubed) of the features, allowing it to fit curved relationships and capture such non-linear price trends that a straight line would systematically under- or over-predict. This added flexibility increases model complexity and, if the polynomial degree is set too high relative to the amount of training data, the model can overfit — closely tracking noise in the training set and producing wild, unrealistic predictions for house sizes outside the training range (poor extrapolation). Interpretability also degrades as polynomial degree increases: while a linear model's single coefficient per feature is easy to explain to a client, a high-degree polynomial's combined effect

on price is much harder to describe intuitively, even though its in-sample fit may look better. In practice, a low-degree polynomial (degree 2 or 3) combined with regularisation is often a good compromise between capturing genuine non-linearity and avoiding overfitting.

Q13. Compare K-Means Clustering and DBSCAN using a dataset with irregular cluster shapes. Analyze differences in cluster detection, handling of noise and outliers, requirement of parameters, and scalability.

Answer:

On a dataset with irregular, non-spherical cluster shapes (e.g. geographic data with elongated or crescent-shaped groupings), K-Means performs poorly because it assumes clusters are roughly spherical and equally sized around a centroid, often splitting a single irregular cluster into multiple pieces or merging separate ones. DBSCAN (Density-Based Spatial Clustering) instead groups points that are densely packed together and can therefore detect clusters of arbitrary shape, following the contours of dense regions rather than assuming any particular geometry. A major practical difference is how each method handles noise: K-Means forces every point into some cluster, even outliers that don't naturally belong anywhere, which can distort centroid positions, whereas DBSCAN explicitly labels sparse, isolated points as noise and excludes them from any cluster, making it inherently more robust to outliers. In terms of parameters, K-Means requires the number of clusters k to be chosen in advance, which can be difficult to determine for unknown irregular structure, while DBSCAN instead requires a neighbourhood radius (ϵ) and a minimum-points density threshold, which can also be non-trivial to tune but does not require knowing the number of clusters beforehand. Regarding scalability, K-Means is generally faster and scales roughly linearly with data size, whereas DBSCAN's naive implementation can be slower (though spatial indexing structures like KD-trees bring it close to K-Means' efficiency on lower-dimensional data), making the choice between them a trade-off between assumed cluster shape/noise handling and raw computational speed.

Q14. Compare Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks for time-series prediction tasks such as stock price forecasting. Analyze differences in handling long-term dependencies, vanishing gradient problem, computational complexity, and prediction accuracy.

Answer:

For stock price forecasting, a standard RNN processes the time series sequentially, maintaining a hidden state that is supposed to carry information from earlier time steps forward, but in practice it struggles to retain information over long sequences because repeated multiplication of gradients during backpropagation-through-time causes the vanishing gradient problem — gradients from distant past time steps shrink toward zero, so the network effectively 'forgets' long-term dependencies and relies mostly on recent price movements. LSTM networks address this directly by introducing a gated memory cell with input, forget, and output gates that regulate what information is added, retained, or discarded at each time step, allowing gradients to flow through the memory cell largely unimpeded across many time steps; this lets LSTMs capture longer-term trends and dependencies (e.g. a pattern that unfolds over several weeks) far more effectively than a plain RNN. This added capability comes with higher computational complexity, since each LSTM cell has several gates with their own weight matrices, requiring more parameters and computation per time step than a simple RNN cell, and consequently longer training times. In terms of prediction accuracy, LSTMs generally outperform plain RNNs on time series with meaningful long-range dependencies, such as stock prices influenced by trends spanning many trading days, while a plain RNN may perform adequately (and train faster) on tasks where only short-term, recent dependencies matter.

Q15. Compare Naïve Bayes and Support Vector Machines (SVM) for classification problems such as sentiment analysis. Analyze differences in assumptions, decision boundaries, computational efficiency, and performance on high-dimensional data.

Answer:

For sentiment analysis on text reviews, Naïve Bayes assumes that word features are conditionally independent given the sentiment class, which is not literally true (word choice and phrasing are correlated) but allows probabilities to be estimated extremely quickly via simple frequency counting, making it very fast to train and update, particularly on large text corpora. SVM makes no such independence assumption; it instead searches for the maximum-margin hyperplane (or, with a kernel, an implicit non-linear boundary) that best separates positive and negative sentiment examples, which can capture more nuanced combinations of words but requires solving a more expensive optimisation problem during training. In terms of decision boundaries, Naïve Bayes' boundary is implicitly defined by the ratio of class-conditional probabilities and tends to be simple, while a linear-kernel SVM explicitly maximises the margin between classes, often generalising better near the boundary where ambiguous, borderline reviews lie. On high-dimensional, sparse data such as bag-of-words or TF-IDF text features, both methods tend to perform well, but SVM (especially with a linear kernel, which is standard for text) often achieves higher accuracy because it directly optimises for class separation rather than relying on an independence assumption, at the cost of being slower to train on very large datasets and less naturally suited to online/incremental updates compared to Naïve Bayes' simple additive counting.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand